Q22 FLIGHT LAB

Architecture, diagnostics and Codex implementation handoff

Revision 1.0 | 3 October 2026

Local-first. Evidence-linked. Built for aircraft troubleshooting and measured performance improvement.

A complete product and analysis specification for a cross-platform desktop app covering Q22 flight-controller, ground-station, radio and related evidence.

Includes architecture, collection and recovery workflows, 28 diagnostic rule requirements, firmware-aware analysis logic, UX, implementation stages and acceptance criteria.

DESIGN DELIVERABLE - APPLICATION IMPLEMENTATION IS DEFERRED TO CODEX

Primary-source facts are referenced. Product decisions, algorithm proposals and performance targets are explicitly design requirements rather than claims of completed implementation.

Reading guide

Start with sections 01-03 for the decision and scope. Codex should then read the complete specification and the companion implementation contract.

01-06 Product and architecture

Decision, Q22 identity, coverage, stack, boundaries and data model

07-10 Collection and interpretation

One-button workflow, adapters, clocks and recommendation policy

11-18 Diagnostic methods

Vibration, filters, PID, power, propulsion, navigation, links and live alerts

19-23 Product delivery

UX, controlled experiments, milestones, validation and required evidence

24-26 Implementation research

Firmware field/parameter catalog, logging profiles and symptom workspaces

Companion files

Q22_Codex_Specification.txt is the complete searchable text. Q22_Implementation_Contract.json contains machine-readable architecture, entities, rules and acceptance families. CODEX_START_HERE.txt provides the implementation brief.

Research boundary

No actual Q22 flight BIN/TLOG/RF bundle was analyzed for this design. Device integrations, parameter compatibility and numerical tuning methods still require the defined fixture and hardware validation. No app code is included.

01 Decision and scope

Build Q22 Flight Lab as an offline-first native desktop application for Windows, macOS and Linux. Use a Tauri 2 shell, React/TypeScript interface, and a packaged Python analysis engine. This is a design and research handoff for Codex, not an application release.

Decision	Selection	Reason / boundary
App type	Local native desktop	Large logs, USB/SD access, offline field use and background recorders.
Initial platform	Windows x64 first; Linux x64 and macOS next within the same architecture	Matches the Mission Planner ground station; cross-platform installers remain a release requirement.
Main action	Collect flight logs	A configured collection job with per-source outcomes, not an indiscriminate disk/network scan.
Tuning authority	Review and export proposals only	No automatic parameter writes, arming, mode changes or flight commands.
Data location	Local by default	No mandatory cloud upload, account or live internet dependency.

The product has two linked workflows: record and monitor available health signals during a flight; collect and deeply analyze the complete flight evidence after landing. The ground station remains responsible for flight control. The app explains measurements, identifies problems and proposes controlled investigations or tuning candidates. It does not replace ArduPilot failsafes or directly control the aircraft.

Make collection simple after initial setup: Start flight recording preserves short-lived sources; Collect flight logs finds local/SD-card logs, retrieves accessible completed controller logs, matches the records to a flight, and explains every unavailable source. A post-flight button cannot recreate telemetry or radio history that was never recorded.

The preferred user experience is an installed app with its runtime bundled. Users should not need Python, Node, a terminal, an API key or a cloud account. Tauri supports packaged external binaries; platform-specific building and validation are required [R01]. A hosted-only website is a poor primary fit for unrestricted log sizes, unattended local collection and native device access.

The analysis core is deterministic and versioned. Optional language-model explanations can be added later, but measured values, fault classification and parameter proposals must come from inspectable algorithms and source evidence. An AI-generated narrative is never a substitute for a validated diagnostic rule.

This specification is intentionally broader than a simple graph viewer. A complete release includes data-quality checks, aircraft health, control/filter analysis, energy comparisons,

collection recovery, issue tracking, immutable tuning baselines and explainable live alerts. Features awaiting actual device samples must be labeled unvalidated or unsupported, not represented by synthetic success.

02 Q22 context and configuration identity

Preserve the following project facts as profile inputs. Values reported in earlier work are not substitutes for current firmware readback or measurements.

Area	Known context	Required handling
Primary aircraft	Cube Orange+; Airbot Mini Carrier PRO v2 / PDB Pro v2; ArduCopter; AUX2-AUX5 / SERVO10-13	Expected ESC log instances 9-12 from the existing review; confirm by individual motor identification on installed firmware.
Alternate aircraft	DEF Drones H7 Basic v3; Matek FCHUB-12S v2; separate supply/regulator arrangement	Separate profile and firmware metadata; expected output/ESC instances 0-3 require confirmation. Do not apply Cube assumptions to this build.
Propulsion	APD 120F3[X]v2; T-Motor MN605-S KV170; NS22x6.6 props; quad X configuration	Store motor/prop revision, rotation, physical corner, servo output, ESC instance and telemetry capabilities.
Energy	Reported 12S 40 Ah Xingto pack	Battery identity, chemistry, manufacturer limits, measured usable capacity, sensor calibration and temperature are required before limit/endurance advice.
Navigation	Here4; established installation/orientation	Capture device IDs, orientation, active sensor lanes, calibration and firmware. Same mounting does not justify copying sensor calibration offsets.
Ground/control	Mission Planner on Windows; QGroundControl may also be used; GX12 USB joystick	One controlling GCS. Separate received telemetry, operator inputs and actual onboard response.
Radio/video	DigiQ SkyPulse air and ground; SIYI A8 mini; 5.8 GHz RHCP/LHCP installation	Record actual radio/camera versions, antenna/cable changes, interfaces, codec and stream measurements. Do not assume a codec or undocumented metric.
Tune history	Keep original Q22 settings and the Q22 configuration incorporating Brian's tune	Maintain immutable named baselines and actual per-flight readback snapshots, plus independent candidate versions.

Name the protected baselines “Q22 original - before Brian settings” and “Q22 with Brian tune”. Preserve Brian's original source file separately as provenance. Never manufacture the

missing pre-Brian baseline from a guessed inverse diff. Resolve the exact accepted files before implementation seeds those labels.

Reported target firmware includes Cube 4.7.1-era work and a separate H7 custom build. Identify each flight from logged version/board information, AUTOPILOT_VERSION where available and the actual parameter snapshot. Pin matching metadata and source revision. A filename containing a version is a hint, not verified firmware identity.

Carry unresolved hardware commissioning items from the existing manual into a hardware-evidence register. Logs cannot establish connector polarity, capacitor voltage suitability, power-rail ripple or correct physical motor direction. Record the observation, measurement, instrument, photo/document reference and closure evidence separately.

03 Product requirements and coverage contract

“Monitor everything important” means every relevant subsystem is represented, with its evidence status and limits visible. It does not mean declaring a subsystem healthy when the necessary sensors or logs do not exist.

Domain	Required result	Examples of unavailable evidence
Data integrity	Identity, source coverage, gaps, corruption, duplicates, reboot boundaries, clocks, logging performance	A TLOG cannot replace high-rate onboard data or reconstruct downlink losses.
Vibration / IMU	Per-sensor vibration, clipping, gyro spectra, sensor errors and redundancy	VIBE is not a frequency spectrum; a post-filter trace cannot show unlogged pre-filter energy.
Control / tuning	Attitude/rate/position tracking, lag/overshoot, PID terms, saturation, filter trade-offs	Gain advice is withheld without representative excitation, noise evidence and actuator context.
Power / propulsion	Pack power/energy, voltage under load, ESC electrical/thermal/RPM data, motor balance and output demand	No individual-cell diagnosis without cell data; no thrust efficiency without thrust or a validated model.
Navigation / estimator	GPS quality, estimator innovations/status, lane changes, heading consistency and position/height response	A good GNSS fix alone cannot prove trustworthy navigation.
Control path / failsafes	Pilot command, GCS receipt/transmission, onboard input and response, mode/failsafe transitions	RCIN alone does not prove the GX12 USB input reached the GCS correctly.

Domain	Required result	Examples of unavailable evidence
RF / network / video	Both radios, HTTP collection health, link/throughput indicators, GCS network and video diagnostics	RSSI alone does not identify interference or explain a video freeze.
Controller / GCS system	Scheduler/load/logging health, resets, rails, disk space, PC sleep and recorder interruptions	Low-rate telemetry may miss a transient reset cause or short electrical event.
Configuration / experiments	Firmware-aware parameter diff, hardware revision, baseline regression, matched-condition comparisons	One changed flight does not prove a change caused an improvement.

Each domain has five independent concepts: evidence availability, data quality, observed condition, diagnostic confidence and issue workflow state. Do not combine them into one aircraft-health percentage. A coverage indicator can describe the fraction of required evidence present, but its label must say evidence coverage and expose its denominator.

All findings must be reproducible from a source hash, parser/rule version, aircraft-profile snapshot and selected interval. The system must retain rejected or incomplete inputs in the collection manifest and explain why a metric or recommendation was withheld.

04 Architecture and technology decisions

Use a modular desktop application, not a collection of separately launched scripts. Keep native access, numerical work and presentation behind explicit contracts.

Component	Decision	Responsibility
Desktop shell	Tauri 2 / Rust	Window, native file dialogs, allowed paths/devices, OS credentials, process lifecycle and packaging.
Interface	React + TypeScript + Vite	Stateful working surfaces, collection progress, findings, comparisons and accessible navigation.
Charts	Apache ECharts; narrow series payloads	Linked time-series, spectra, spectrograms, scatter plots and event overlays. Benchmark rather than assuming advertised point counts [R03].
Engine	Packaged Python with pymavlink, NumPy, SciPy and PyArrow	Decoding, units, segmentation, DSP, diagnostics, local collectors and report generation.

Component	Decision	Responsibility
Metadata	SQLite, single writer	Profiles, sessions, manifests, alignment records, findings, experiment history and job state.
Numeric store	Partitioned Parquet; DuckDB queries	Out-of-core column selection and time-window queries. Parquet filter/projection pushdown supports this design [R02].
IPC	Typed request/event envelopes over supervised stdio	Renderer invokes limited Rust commands; Rust routes to engine. No production HTTP server is required.
Packaging	Per-OS CI builds with bundled engine	Windows installer, macOS app/DMG, Linux AppImage and/or supported package. Sign/notarize release builds where required.

The shell supervises one engine coordinator. The coordinator owns catalog writes and schedules bounded parser/analysis worker processes. Live recorders run independently of heavy FFT/import jobs, with queues and backpressure. A large post-flight import must not starve live recording. If the UI closes during recording, offer Keep recording in tray, Stop and close, or Cancel; never silently abandon a recording.

Use newline-safe framed IPC or JSON-RPC with explicit request IDs, protocol version, cancellation, progress and errors. Route diagnostic logging to stderr so it cannot corrupt protocol messages. Return references and bounded chart slices, not entire multi-gigabyte arrays over IPC. Handle sidecar crash/restart without losing completed archives or presenting stale values as live.

A local HTTP API can exist as a development-only alternative. If added in production later, bind loopback only and enforce per-launch authentication, strict origin/host checks and narrow endpoints. It should not be necessary for the first installed app.

Lock dependency versions only after the first cross-platform packaging spike. Do not copy exploratory environment versions from this conversation into a release. Verify Python wheels, native dependencies, webview behavior, serial access and filesystem permissions on clean Windows/macOS/Linux machines.

Electron is a reasonable fallback if Tauri's platform webviews block required interaction or packaging, but it adds a bundled browser runtime. Python/Qt is viable for a team prioritizing one implementation language; it is less aligned with the proposed rich web-style chart workspace. A PWA is useful as a future report viewer, not the main collector. The recommended choice remains Tauri plus Python.

05 Internal boundaries and reproducibility

Separate collection, decoding, interpretation and advice so new log formats or firmware revisions cannot silently alter old conclusions.

The engine should also provide a headless CLI with the same analysis core for repeatable batch tests and engineering review. The desktop app remains the normal user entry point. CLI and UI must not have different numerical implementations.

Before reusing FilterReview/PIDReview or other third-party code, inventory licenses and notices for every file and submodule. ArduPilot WebTools identifies GPL-3.0 licensing; decide a compliant reuse/distribution approach rather than assuming it is permissively licensed. Pin commits and document intentional changes. A rewritten implementation still needs independent numerical validation [R07].

Use a content-addressed raw store (SHA-256) and immutable per-flight manifests. Stream to a temporary file, flush, hash and atomically commit; preserve file size and source metadata. Hash equality deduplicates bytes, not necessarily flight identity. Different source devices may legitimately contain overlapping observations.

Store data tables by flight/source/boot epoch/message group, with explicit schema versions. Do not load every message from a large raw-gyro log into Python dictionaries. Parse in batches, write Parquet, query only required columns/windows and compute plot decimation separately from metric calculation.

Never execute commands or render active markup from a filename, STATUSTEXT, radio response or imported note. Validate archive extraction paths, input sizes and decoder resource limits. Plugin adapters are trusted installed code; a log is data, not a plugin.

- Collector adapter: discover, probe capabilities, start/stop recording, list completed records, retrieve, resume and report a recovery action. Its output is original bytes plus a manifest; it does not invent engineering units.
- Parser adapter: sniff file structure, decode messages, retain source identity and raw timestamps, emit schema/unit metadata, records and corruption/truncation reports. Use file content as well as extension.
- Normalizer: map validated fields into typed signals, apply scaling exactly once, preserve instance/output mapping and mark missing/sentinel values explicitly.
- Clock service: create boot epochs, estimate or accept time transforms, retain anchors/residuals/uncertainty and refuse unjustified synchronization.
- Segmenter: propose armed/airborne/hover/cruise/climb/descent/turn/transient phases with confidence; let the user correct them. Segments have versioned definitions.
- Metric service: compute descriptive results using validated units and eligible intervals. Return denominator, data coverage, uncertainty and excluded intervals.
- Rule engine: evaluate versioned prerequisites, findings and hypotheses. Rules are deterministic, testable and explicitly distinguish published thresholds from engineering heuristics.

- Recommendation planner: convert supported findings into inspection, collection, configuration review or a bounded experiment. It must abstain from a parameter value when the evidence is insufficient.
- Report service: freeze the input manifests, versions, figures, findings and limitations into portable JSON plus human-readable HTML/PDF. Regeneration does not overwrite earlier reports.

06 Core data model

The machine-readable contract in this handoff defines the same entities. Add schemas and migrations before implementation relies on them.

Entity	Required contents
AircraftProfileVersion	Aircraft ID; board/firmware identity; physical motor map; ESC instance mapping evidence; motors/props/battery; calibration status; hardware changes; limits and their provenance.
FlightSession	UUID; aircraft profile version; operator/test notes; mass/payload/battery/weather; actual parameter snapshots; source IDs; proposed and confirmed flight intervals.
SourceManifest	Original name/location; SHA-256; byte size; device and software identity; acquisition method; start/end clocks; parser version; success/partial/error status; warnings.
SignalSchema	Source message/field; physical quantity; raw and engineering units; scaling provenance; sensor/ESC/system/component instance; expected and observed cadence; sentinels.
Sample / event	Source ID; boot epoch; source sequence/message offset; raw time; time domain; normalized time if valid; value or null; quality flags.
TimeAlignment	Source and target clocks; anchor pairs; affine/piecewise transform; offset/drift; fit residuals; uncertainty; confidence; user adjustments and provenance.
MetricResult	Algorithm/version; interval and phase; value/unit; count and time coverage; exclusions; uncertainty; source/field references; interpretation limits.
Finding	Rule/version; severity; evidence quality; confidence; observation; competing hypotheses; evidence intervals; next action; expected result; parameter applicability; retest criteria.
ParameterSnapshot / proposal	Immutable original; parsed values; firmware metadata; before/after diff; reason; target aircraft; validation status; rollback snapshot; no implicit application.
Experiment / issue	Baseline and candidate; one functional change; confounders; planned success criteria; repetitions; results; open/investigating/retest/resolved history.

Entity	Required contents
CollectionJob	Per-adapter state/progress; resumable transfer position; retries; chosen flight grouping; missing-data consequences and recovery instructions.

Use integer microseconds/nanoseconds for persisted clock values. Preserve UTC epoch precision: serialize 64-bit absolute times as decimal strings over JavaScript-facing JSON, or use a typed binary format; use floating relative seconds only for plotting. Null means unavailable. Zero is reserved for a real decoded zero with valid quality.

Preserve original message definitions, FMT/FMTU units/multipliers and the parser's scaling behavior. Store both the original field reference and the normalized quantity. Unknown units remain unknown. A firmware upgrade can require a new mapping, not an automatic application of the previous scale.

SQLite is the catalog and issue history; Parquet holds large numeric arrays. DuckDB is a local query engine over those tables, not a second competing metadata authority. Limit concurrent writers and make migrations recoverable.

07 Collection experience and recovery

Configure once, then use two clear buttons: Start flight recording and Collect flight logs. Collection is a coordinated job whose partial results remain useful.

Failure	User-facing recovery
Controller port busy	Disconnect the GCS from that USB port, retry, or use its downloader/SD card. Preserve the controlling GCS during flight.
Aircraft armed / active log growing	Defer the onboard transfer until a completed log is available. Keep live recording active.
No TLOG found	Choose the actual GCS log directory; verify recording before the next flight. Do not claim recovery of packets never logged.
RF history absent	Explain that live capture was required; test the configured endpoints and start recording next time.
Unknown vendor format	Archive it, identify device/firmware and request one representative export/schema. Give manual export instructions when documented.
SD card or disk full / sleep interruption	Show the exact interrupted interval, preserve completed data and explain the recording gap.
No optional camera/system log	Keep flight analysis available; mark camera/system conclusions unassessed.

- First use: select aircraft/profile; choose approved Mission Planner/QGC and camera folders; identify controller USB device; enter radio addresses; confirm optional forwarded telemetry. Each optional source may be skipped.
- Start flight recording: verify disk space, clock status and recorder reachability; open a session envelope; begin configured ground/RF/system recorders. Show live sample ages and whether each source is actually producing records.
- Collect flight logs: scan configured folders and mounted SD log directories; query a configured available controller after disarming; include RF/system captures; copy selected records to the raw archive; hash and validate each transfer.
- Match records using aircraft/system IDs, boot identity, time anchors and file intervals. File modification time alone is weak evidence. Propose ambiguous groupings and ask the user to choose; do not merge unrelated flights merely because they were imported together.
- Run completeness checks before deep analysis. Show each source as collected, partial, still recording, unavailable, unsupported, failed or not configured. State which analyses are blocked and provide a source-specific next action.
- Resume an interrupted transfer only after confirming device/log identity and consistency. Never mark a growing or incomplete remote log complete. Retain successfully collected sources if another adapter fails.
- Watch configured directories only after the user enables auto-collection. Detect stable file size/closed recordings before ingestion; treat open TLOGs as provisional. Avoid full-disk scanning, uncontrolled network sweeps or opening every serial port.

08 Source adapters and hardware capabilities

Implement adapters as capability-driven integrations. “The model supports a feature” and “this installed firmware produced that measurement” are different facts.

Adapter	Required behavior	Evidence boundary
ArduPilot DataFlash	BIN and textual DataFlash LOG; FMT/FMTU-aware schema; parameters, events and raw/batch sensor records	Decode supported firmware schemas; retain unknown message types and report undecoded fields.
GCS MAVLink	Mission Planner/QGC TLOG and documented forwarded stream; preserve system/component IDs and receive timestamps	Separate GCS-sent, vehicle-sent and routed/duplicate messages where the capture allows it.
Controller download	Read-only LOG_REQUEST_LIST/DATA/END baseline; optional tested MAVFTP acceleration; SD import fallback	Capability-probe firmware and paths. Do not require newer virtual paths on a 4.7.1-era controller [R11, R12].

Adapter	Required behavior	Evidence boundary
SkyPulse air/ground	Configured read-only statistics/events polling with raw responses, UTC and monotonic collection times	Manual endpoints expose rolling history. Firmware response structure and units must be verified from actual samples [R10].
APD/ESC	Consume available onboard/telemetry RPM, electrical, thermal and error fields; record unsupported channels	Plain BDShot RPM is not evidence of electrical/thermal coverage. Extended telemetry capability depends on ESC/firmware [R09].
Parameter exports	Mission Planner name/value and QGC formats; whole snapshots plus logged changes	Snapshot/readback identity matters; logged startup parameters may not establish an entire later flight if settings changed.
EdgeTX / GX12	Optional enabled SD Logs CSV import from the radio's mounted storage after flight	USB joystick operation does not guarantee aircraft telemetry is available on the radio. Do not change USB mode during flight [R31].
SIYI A8	Local media/sidecar import and documented Ethernet download where supported; camera metadata and events	Media retrieval does not imply durable diagnostic logs or measurable end-to-end latency [R19].
Ground computer	Optional local timestamped recorder for disk, CPU, memory, network and process/recorder health	OS-specific capability and permissions must be tested; no hidden claims of GPU decoder or joystick visibility.
Generic exports	CSV/JSONL mapping wizard with explicit units, clocks and instances	Never guess whether RSSI is dBm, a percentage or a vendor scale. Save the mapping with firmware/version.

For SkyPulse, the documented read-only GET endpoints are /scatter_data.json and /events.json, with about 50 seconds of history. Preserve every poll and failure. Dataset windows overlap; derive deduplicated samples only after validating the exact sample identity/time schema. Collection time bounds when the reply was observed, not when every point inside it occurred. A five-second poll interval is a proposed starting point to bench-test, not a guaranteed vendor requirement [R10].

Do not compete with Mission Planner/QGC for a serial connection or silently alter MAVLink stream rates. Prefer the existing GCS TLOG for post-flight review; use explicitly configured forwarding/router duplication for the app's live monitor. Record router configuration and traffic counters. Log download should be deferred while airborne and should not saturate the control radio link.

No automatic flight command, parameter write, log deletion or firmware update belongs in the collector API. Read-only parameter/version requests and log transfers must be explicit capabilities with tests that inspect all outbound MAVLink messages.

09 Time alignment and flight segmentation

A single pane of glass needs a shared timeline when evidence supports one, and visibly independent clocks when it does not.

Preserve these domains independently: FC boot/sample time; GPS week/time-of-week or validated UTC; TLOG ground receive time; PC wall clock and monotonic clock; radio-internal time; camera media time. Split at FC reboot, clock discontinuity, capture restart and counter wrap. Never sort two boot epochs into one apparent continuous flight.

Preferred alignment anchors are valid logged GNSS time paired with FC time, and MAVLink SYSTEM_TIME with valid Unix/boot values. Validate leap-second handling, time validity and source system/component. Estimate a piecewise affine transform from multiple anchors when drift matters. Store the chosen anchors, method, residual distribution and uncertainty [R13].

TLOG receive time includes link and scheduling delay. Shared arming/mode events are fallback anchors with bounded uncertainty, not proof of millisecond synchronization. Cross-correlation is a proposed alignment aid and must be labeled estimated; correlation is not causal attribution. Radio timestamps with undocumented epochs remain unaligned until verified.

Show alignment uncertainty as a range when users compare events across sources. Suppress claims that require ordering finer than the uncertainty. A statement such as “the RF drop preceded the rate error by 40 ms” is invalid if alignment uncertainty is 500 ms.

Detect armed intervals separately from airborne intervals. ARM or heartbeat state alone is not takeoff. Propose takeoff/landing from multiple available signals such as altitude change, vertical velocity, output demand and landed state. Uncertain phases must remain editable and visible.

Classify hover/cruise/climb/descent/turn/transient phases using recorded motion, commands and mode, with configurable thresholds and minimum dwell time. Initial engineering defaults can be proposed and tested, but are not universal ArduPilot limits. Store the segmentation algorithm/version and user corrections.

All metrics must state their interval and exclusions. Do not count disarmed bench time as flight energy, bridge gaps in integrated power, mix pre/post-parameter-change segments, or compare aggressive maneuvering against steady hover as an efficiency result.

10 Diagnostic policy and recommendation levels

Every insight begins with an observation and can advance only as far as the evidence allows.

Level	What the app may say	Required support
Observation	“IMU 1 clipping increased during this interval.”	Valid decoded samples, counter handling, exact source/time references.

Level	What the app may say	Required support
Hypothesis	“Mechanical vibration is a plausible contributor.”	Multiple relevant observations and competing explanations; confidence stated.
Investigation	“Inspect the mounting, then capture pre/post gyro data.”	Action addresses an evidence gap or plausible cause; expected result defined.
Configuration review	“Check the selected notch tracking source against actual RPM coverage.”	Exact firmware metadata and actual source capability; no assumed bitmask.
Parameter candidate	“Evaluate this proposed change set in this bounded test.”	Validated model/method, data sufficiency, constraints, before/after values, applicability and rollback.
Confirmed improvement	“Repeated comparable tests improved these measured outcomes.”	Predetermined metrics, matched conditions, repetitions and uncertainty; no unexplained regressions.

Severity and confidence are separate. A potentially serious but poorly evidenced condition can warrant investigation without claiming its cause. Use an evidence gap state for missing data, not a red aircraft-fault alarm. Published limits, hardware limits, user acceptance criteria and engineering heuristics must be labeled distinctly.

Each rule specifies eligible phases, minimum coverage/sample rate, debounce/dwell, hysteresis, prerequisites, exclusions, reference and firmware applicability. If a prerequisite fails, produce a specific “cannot assess” result and collection/test guidance. Avoid arbitrary global health scores.

Recommendation cards show the observation, source/fields/instances, time interval, confidence, possible causes, proposed action, expected benefit, possible trade-offs and a retest criterion. A parameter name listed as relevant is not itself a recommendation to change its value.

Do not infer physical RF packet loss from MAVLink sequence gaps alone, exact motor damage from asymmetric current alone, PID direction from high current alone, or a notch center from a VIBE summary. A diagnostic model must be validated against representative labeled fixtures before its conclusions are promoted beyond exploratory status.

11 Vibration, clipping and IMU analysis

Prioritize mechanical and sensor health before gain optimization. Keep each IMU instance visible.

Decode the actual VIBE schema. Modern per-IMU records and older Clip0/Clip1/Clip2 layouts require different mappings. Compute axis distributions, p50/p95/p99, maxima and time spent above the published 30 and 60 m/s² guidance bands. Use time weighting for exposure and sample quantiles for distribution; label each accurately. Brief landing impacts must remain distinguishable from persistent airborne vibration [R04, R14].

Clipping values are counters: compute positive increments within each continuous boot/sensor epoch. A counter decrease is a reset/wrap/format event to resolve, not a negative clipping event. Do not attribute the initial counter value to a selected interval. Report count, rate, onset, duration and flight phase; show individual sensors before an optional aggregate.

Correlate vibration with motor RPM/output demand, speed, commanded angular motion and aircraft configuration only after alignment is established. Inspect repeated RPM orders, speed-related growth, loose mounting/cable contact, propeller balance/tracking and motor bearing candidates. These associations narrow inspection; they do not prove the mechanical cause.

Compare redundant IMUs using calibrated quantities, instances and health/error records. Check sensor disagreement, clipping, sampling discontinuities, temperature association, active primary/lane changes and unexplained estimator divergence. Do not average away a defective sensor or merge data from different sensor orientations.

If persistent clipping or major mechanical vibration exists, defer optimization proposals that could hide the symptom. The next step should prioritize inspection, sensor installation verification and a comparable confirming log. A notch filter is not a remedy for accelerometer saturation.

Acceptance: fixtures must cover high-amplitude brief impact, sustained airborne vibration, older/newer VIBE schemas, a reset inside the file, missing one IMU, unknown scale, clipped signal with absent VIBE, and a selected window beginning after an existing counter increment.

12 Gyro spectra and harmonic-notch analysis

Implement proper raw-gyro analysis, including raw and batch formats. Frequency analysis of the low-rate VIBE envelope is prohibited.

Support eligible GYR records and ISBH/ISBD batch data through firmware-aware decoders. Reconstruct batch sequence, instance, sensor type, scaling, sampling rate and sample time; detect missing/duplicated batches. Preserve pre/post-filter identity. Verify this against a pinned ArduPilot FilterReview implementation and known fixtures; do not merely infer format from a field name [R05, R07].

Within stationary-enough segments, remove the mean or documented trend, apply a specified window and compute Welch PSD with recorded sample rate, FFT length, overlap, resolution and units. Report amplitude spectra and power spectra distinctly; use 20 log10 for amplitude ratios and 10 log10 for power ratios. Validate Parseval/known-sine fixtures. Do not resample across large gaps or claim resolution beyond the data duration.

Expose per-axis/per-IMU PSD, time-frequency spectrograms and RPM-order overlays. Convert verified mechanical RPM to rotation frequency using RPM/60. Electrical RPM or a pole-pair-scaled field requires the correct telemetry conversion first. Never apply motor-pole scaling twice. Blade-pass orders require the actual blade count, not an assumption.

Compare pre/post-filter noise only for matching sensor, phase, sample provenance and scale. Calculate band power reduction and residual peaks. A lower plotted post-filter peak is not automatically better if filtering adds harmful control-band delay or the flight conditions differ.

Candidate notch planning checks source coverage and freshness, whether dominant peaks track RPM/throttle/FFT, relevant harmonics, width, attenuation, minimum-frequency behavior, phase delay and controller load. Prefer the smallest supported filter set that addresses repeatable noise. Model with the exact firmware filter equations and parameters, including a second notch bank and dynamic options where available [R06].

Relevant parameter families include INS_HNTCH_*, INS_HNTC2_*, INS_GYRO_FILTER, gyro/raw logging configuration and applicable ESC/RPM source settings. Resolve each name, range, bitmask and reboot requirement from the installed firmware. Do not copy wiki numeric masks when prose, versions or source definitions disagree.

A numeric candidate requires verified spectral/RPM data and firmware-correct simulation. Show predicted attenuation and delay as modeled results, then require a confirmation flight with measured residual spectra, control response, CPU/logging health and temperatures. If raw data or RPM is insufficient, recommend the exact missing capture rather than invent a center frequency.

13 Attitude, PID and actuator diagnosis

Evaluate the control chain: pilot/navigation demand, desired attitude/rate, achieved motion, PID contributions and available actuator authority.

Use ATT, RATE and PIDR/PIDP/PIDY where supported, plus actual output/motor-limit evidence. Compute phase-specific error RMSE, absolute-error quantiles, bias, lag, rise/settling time, overshoot, residual oscillation bands and repeatability. Rate-only records support fewer conclusions than full PID/output/noise evidence [R08, R14].

Step-response metrics require a suitable isolated command change, adequate amplitude, initial/final steady behavior, known sampling and enough post-command time. Use reference normalization and defined rise/settling bands. Reject overlapping commands, saturated maneuvers, clipping, major estimator resets and data gaps. Fit or average responses only across comparable maneuvers.

Estimate lag with a documented method and uncertainty; cross-correlation of highly periodic or unexcited data can be ambiguous. Frequency-response/coherence analysis requires sufficient excitation and correct interpretation of closed-loop input/output signals. Do not report gain/phase margins from an invalid transfer estimate.

Separate undertracking due to command shaping, actuator saturation, insufficient thrust, filter delay, wind/disturbance, estimator issues and gain selection. Inspect I accumulation/recovery, P/D activity, feedforward, Dmod/SRate and applicable flags. A high D-term RMS can reflect sensor noise, commanded motion or gain behavior; it is not a standalone instruction to reduce D.

Map RCOU/output channels through SERVOx_FUNCTION and the verified motor order; protocol and configured endpoints matter. PWM-like numerical bounds are not universal across DShot/scaled outputs. Use firmware motor-limit/allocator flags when available, and label output-near-bound heuristics as such. Distinguish total thrust, attitude/yaw authority and individual output limitations.

Parameter families to review include the applicable ATC_RAT_RLL_*, ATC_RAT_PIT_* and ATC_RAT_YAW_* gains/filters/limits, ATC_ANG_*, ATC_ACC_R_MAX, ATC_ACC_P_MAX, ATC_ACC_Y_MAX, ATC_INPUT_TC and relevant motor parameters. Presence and naming are firmware-dependent. Do not issue a generic recipe to increase P, reduce D or remove filtering.

A tuning plan first resolves sensor/mechanical/estimator issues, then validates filtering, then uses an appropriate controlled tuning method or ArduPilot AutoTune workflow with actual vehicle constraints. Any app-generated numerical gain candidate must be supported by a validated system-identification/optimization method; otherwise offer a test plan and evidence links. Preserve the previous accepted parameter snapshot and change one functional group at a time.

14 Power, battery and endurance optimization

Energy analysis must distinguish measured consumption, comparable performance and a causal explanation for improvement.

Calculate simultaneous electrical power $P(t)=V(t)\times I(t)$. Integrate signed power over valid continuous intervals: $E_{Wh} = \text{sum}[0.5(P_i+P_{i+1})\times dt]/3600$. Select a declared primary battery monitor; do not sum an aggregate monitor with the same pack's constituent monitors or combine BIN/TLOG duplicates. Cross-check logged cumulative consumption only after validating its units/resets.

Retain negative current as a measured value and flag calibration/sign/returned-energy interpretation. Never silently clamp it or fill missing values with zero. Define gap tolerance from actual cadence and expected rate; exclude long or invalid intervals and show usable coverage. Distinguish total selected-interval energy from energy in only the observed portion.

Report calibrated pack voltage/current, time-weighted mean and peak power, Wh/min, phase-specific power, voltage under comparable demand, thermal trends and observed usable capacity when a valid discharge test exists. Mean cell voltage from pack voltage divided by cell count is not individual cell voltage or balance.

Hover efficiency comparisons require near-steady height/motion and comparable mass, payload, battery state/temperature, props and wind. Cruise comparisons require defined valid distance/speed and matched segments; report Wh/km only with appropriate overlapping power/navigation coverage. GPS ground speed is not airspeed, especially in wind.

A speed-versus-power curve can identify an observed operating region for lower Wh/km or lower power within the tested envelope. It cannot establish a globally optimum cruise speed from a few unmatched points. Weight-normalized power is a comparison proxy with assumptions, not a direct propulsive-efficiency measurement.

Apparent voltage drop divided by current change can screen pack-plus-wiring resistance only over suitable short segments with near-constant state of charge and temperature; report fit quality and confounders. Do not label it intrinsic cell resistance. Compare pack and per-ESC voltage/current carefully for sensor calibration and timestamp differences.

Endurance estimates are scenarios based on measured/declared usable remaining energy, selected reserve and representative future power. Show assumptions and uncertainty; 40 Ah nominal capacity alone does not establish remaining energy or a safe return time. Do not infer battery chemistry limits from “12S”.

Connect tuning to energy only when oscillation/output activity and power co-occur under comparable conditions, and then frame a hypothesis. Validate by repeated matched flights with tracking/stability/thermal constraints. Lower PID gains can worsen control; lower filter noise can add delay. The objective is a trade-off among stable response, noise, thermal load, authority and energy, not minimum watts at any cost.

15 ESCs, motors, airframe and power distribution

Use the exact physical-output-instance mapping and compare behavior within equivalent operating conditions.

For each configured motor, track available RPM, command, voltage, current, consumed charge, ESC/motor temperature and error/telemetry counters. Report channel completeness and freshness separately. All four RPM values do not establish four temperature/current streams. Check supported extended telemetry and the APD firmware before choosing a collection method [R09].

Analyze motor asymmetry only in stable, comparable phases with known mapping. Control allocation, yaw demands, wind, CG, payload and motor direction can produce legitimate differences. Persistent unexplained asymmetry should generate inspection hypotheses and a bench/mapping/calibration check, not an automatic diagnosis of a failed motor.

Look for abrupt command-to-RPM mismatch, temperature rise inconsistent with demand, telemetry loss, rail/current excursions and repeated output limiting. Require multiple signals and adequate telemetry rate before assigning mechanical/electrical causes. Identify the observed anomaly even when the cause remains unresolved.

Trend airframe/prop changes with vibration orders, phase-specific energy, tracking and durability notes. Logs alone cannot measure frame stiffness, drag coefficient or structural integrity. Add external measurement/test results as evidence rather than inferring them from a single spectrum or endurance number.

Controller, accessory and main battery rails are different. Record the actual regulator/power-module topology for the Cube and H7 profiles. POWR samples can screen logged rail behavior but cannot qualify capacitor ripple, startup transients, connector heating or the reported local capacitor ratings.

Manufacturer operating limits and user acceptance criteria are profile data with a source/version and units. No universal ESC temperature, motor-current or pack-voltage threshold should be hardcoded as safe for every installation.

16 Navigation, heading, altitude and controller health

Troubleshooting should connect observed trajectory/control errors to estimator and sensor evidence without assuming a single cause.

GPS/GPA review covers fix type, satellites, HDOP and available horizontal/vertical/speed accuracy, update gaps, velocity/position discontinuities and receiver changes. Evaluate airborne intervals and required flight modes. A low satellite count or high dilution is a context signal; a reported fix is not a guarantee against interference or spoofing.

EKF review covers lane-specific innovation test metrics, status flags, timeouts, resets, lane/primary changes and aiding changes, linked to GPS, magnetometer, barometer/range and vibration data. Verify version-specific XKF/NKF scales; these test metrics are not raw covariance. For applicable XKF4 quantities, 1 marks the innovation acceptance gate, not the Copter EKF failsafe threshold. Failsafe diagnosis requires the configured FS_EKF_THRESH and matching filtering, multiple-quantity/state/timer logic; prefer explicit recorded activation [R33].

Heading review compares available magnetometers, estimated heading, yaw innovations and current-dependent changes after accounting for orientation and calibration. Association between current and field/heading may indicate interference but does not establish a cause. Do not transplant compass offsets/device IDs from another aircraft because mounting matches.

Altitude/position response uses desired versus estimated/measured motion with correctly decoded frames and sign conventions. Distinguish NED down from upward altitude. Inspect climb/descent command response, vertical oscillation, barometer/range inconsistencies, position-hold error and navigation-controller output limiting.

Controller health includes firmware/board identity, reset/watchdog/crash indications when logged, scheduler timing/load, loop delays, logging drops/buffer pressure, IMU errors, temperature and supply rails. Correlate load increases with high-rate logging, filtering and telemetry. A missing interval may be logger overload rather than a flight dynamics event.

Decode events/error-resolution semantics from the matching firmware. A historic ERR entry is not automatically an active fault; an error-clear code may close an incident. Preserve the event sequence and explicitly distinguish “no resolution captured” from “fault remains present now”.

17 Radio, GCS, joystick and video troubleshooting

Represent the complete observation path: operator input -> GCS/routing -> radio -> onboard receipt -> aircraft response; video uses a separate return path.

Preserve both radios' available RSSI/quality/rate/drop/temperature/Tx-power data and raw API responses. Units, averaging windows, counter behavior and firmware are adapter metadata. Compare directional traffic and RF metrics to application demand without summing overlapping history windows. A local radio API failure may be an HTTP/network failure rather than an RF outage.

For MAVLink, separate each system/component/channel, sequence stream, send/receive direction when known, routing duplicates and signing/version metadata. Report received-stream gaps and message cadence. Sequence discontinuities alone do not measure physical RF packet loss; logger behavior, routing and multiple senders can produce them.

Track pilot input freshness and mode/command requests where available, acknowledgements and actual onboard state. If a command appears on the ground but no onboard response is captured, inspect the missing chain and alignment uncertainty. Neither absence in a partial log nor a COMMAND_ACK by itself proves physical aircraft behavior.

For the GX12 USB path, preserve the EdgeTX model/version and GCS joystick mapping as configuration artifacts. Optional OS/HID diagnostics can observe device disconnects/input timing only if the integration actually exposes them. RCIN records show what ArduPilot received; they are not raw USB joystick measurements.

Video troubleshooting separates source camera health, RTP/transport/network behavior, decoder/render load and display delay. Record codec/resolution/received bitrate/frame/decode-drop data when provided by the actual GCS/player. Avoid launching a second decoder/capture that materially loads the flight computer without an explicit configuration choice.

A recorded video or ping measurement alone cannot establish glass-to-glass latency. Provide a controlled bench latency test workflow with a synchronized visual/time reference. Correlate freezes with radio/GCS evidence only at supported clock accuracy. SIYI media download can be supported separately from diagnostic log availability [R19].

Ground-system health should record process/app version, network adapter/link changes, disk availability, sleep/resume, recorder failures and available CPU/memory/decoder indicators. Alerts must identify when the observer itself is unhealthy so the user does not mistake a stalled dashboard for stable aircraft values.

18 Live monitoring and alert lifecycle

Live monitoring is an additional observer. Its availability and resolution depend on forwarded/recorded telemetry, and it cannot see all high-rate onboard evidence.

- Show aircraft-connected state separately from stream freshness and recorder-active state. Use receipt age and source timestamp/sequence progression; receiving the same old value repeatedly is not necessarily fresh data.
- Define expected cadence per signal/source. Use configurable dwell, hysteresis and clear conditions for alerts. Suggested engineering defaults must be labeled and validated against representative flights, not presented as official thresholds.
- Use eligible-phase rules to avoid a normal disarmed or startup state generating repeated flight alarms. Data gaps, out-of-range values and device faults are distinct incidents.
- Display serious supported observations prominently, with the source age and next diagnostic action. Optional sounds/notifications are user-controlled. Do not replace or modify ArduPilot/GCS alarms and failsafes.
- Deduplicate related symptoms into an incident while preserving individual evidence. Record first/last occurrence, count, active/cleared state, acknowledgement and subsequent recurrence.
- Acknowledge means the user saw it; it does not mean resolved. Only a documented test/retest can close an engineering issue. Preserve investigator notes and the evidence for closure.
- After flight, recompute eligible rules against the more complete onboard data. Label revisions to a live hypothesis and explain why the post-flight conclusion changed.
- On low disk, engine crash, recorder backlog or clock jump, prioritize recording continuity and explicitly mark affected intervals. The last displayed measurement must show stale age, not remain green indefinitely.

19 UX and screen requirements

Use a clean engineering workspace: light main area, restrained navy navigation, blue actions, 16 px body text and at least 14 px regular labels, with progressive disclosure of technical detail.

Screen	Primary content	Essential behavior
Overview	Flight identity, measured summaries, highest-priority findings, timeline and source coverage	Show full-flight versus selected/partial scope. Missing values read Unavailable, not 0.

Screen	Primary content	Essential behavior
Signals	Searchable units/instances; linked cursors; zoom; events; spectra and spectrograms	Display source, clock/alignment confidence and sampling quality. Preserve gaps and decimate only for display.
Insights / issues	Observation, evidence, hypotheses, action, expected result and retest	Click evidence to open the exact chart interval. Filter by subsystem, severity, confidence, phase and workflow state.
Collect	Source setup, Start recording, Collect flight logs, per-source progress and recovery	Useful partial completion; manual import always available; ambiguous flight matches require review.
Compare	Flight/phase metrics and conditions; parameter snapshots and proposed changes	Protected Q22 original/Brian-based baselines, confounder warnings and actual per-flight readback.
Aircraft	Versioned hardware, firmware, mappings, calibration, limits and test notes	Separate declared from measured/verified values. Profile edits do not rewrite historic sessions.

The persistent header shows aircraft, selected flight/session, live/post-flight mode and the main collection action. First launch permits immediate file import without configuring optional integrations. A synthetic demo can be included, but must remain explicitly synthetic in all screens, exports and comparisons.

Default charts show a few related channels with compatible units, not every field on one axis. Provide saved workspaces for vibration, control, power, navigation and link troubleshooting. Selected windows are visible and can be reset; every derived value carries its scope.

An issue follows Open -> Investigating -> Change planned -> Retest needed -> Resolved, with reopen and evidence history. A recommendation is not a one-click “fix”. User notes, accepted/rejected hypotheses and attachments belong alongside the finding.

Parameter comparison must show changed/added/removed values, functional groups, units, bitmask labels, firmware compatibility and calibration/device-specific fields. Never silently apply a snapshot to the connected aircraft.

Support keyboard navigation, visible focus, non-color status labels, screen-reader chart summaries, reduced motion and 200% text enlargement. Test compact ground-station displays as well as large monitors. Empty, loading, offline, stale, corrupt, partial and unsupported states need explicit copy and actions.

20 Parameter proposals and controlled experiments

The app should help the user improve Q22 systematically while preserving a known baseline and documenting trade-offs.

Each candidate change references the actual source snapshot, target aircraft and exact firmware metadata. Resolve parameter availability, range, enum/bitmask meaning, dependencies and reboot requirements. Separate relevant parameters from proposed edits. Reject incompatible boards, unknown fields, mixed firmware assumptions or a stale pre-change snapshot.

Preserve original Q22, Q22-with-Brian and later accepted tunes as immutable versions. A candidate is a branch with a purpose, hypothesis and rollback reference. Same frame/motors/weight/orientation supports considering Brian's tuning as a starting point; it does not eliminate aircraft-specific validation or authorize copying calibration.

Before a test, record the functional group changed, expected metric effect and regression constraints. Capture actual all-up mass, payload/CG, battery ID/state/temperature, hardware/prop changes, wind, flight phase/mode and operator notes. Prefer repeated comparable trials; show missing/confounded conditions rather than normalizing them away.

Compare distributions and time exposure, not only peaks. Report absolute and relative changes with sample/flight counts and uncertainty. Avoid treating thousands of correlated samples from one flight as thousands of independent experiments. Use flight-level or block-aware resampling when estimating confidence.

For endurance-focused work, jointly assess tracking, stability, authority, thermal behavior, noise and energy. A lower-power result that degrades response is not automatically an improvement. Keep the objective and acceptance limits visible.

Export a human-readable change plan and machine-readable proposed diff only after the user reviews it. Applying changes remains a separate deliberate workflow in the established GCS for the initial product. A later write feature would need a separate design and validation decision.

21 Implementation sequence for Codex

Build vertical slices with real parsers and evidence-linked results. Do not substitute a polished mock dashboard for working data flows.

Stage	Deliverable	Exit condition
0 - Foundation	Repository, contracts, dependency/license review, installer/sidecar spike, fixture plan	App/engine handshake and clean install on target OS; raw archive and migration design proven.

Stage	Deliverable	Exit condition
1 - Useful collection and review	BIN/LOG/TLOG/params/CSV/JSONL import; local/SD discovery; RF recorder; manifests; basic coverage/health/power/vibration	Identity/schema/units, per-source boot epochs and explicit scope validated before metrics. Real fixtures reproduce findings; missing/corrupt data never appears healthy.
2 - Full evidence workspace	Cross-source alignment, advanced phase selection, comprehensive health/event channels, issue workflow, comparisons	Correct instance/clock/unit handling and source-linked charts across representative Q22 bundles.
3 - Control/filter/performance	Raw and batch gyro, spectra, filter modeling, PID response and actuator context, matched-condition energy analysis	Parity fixtures versus pinned trusted tools; evidence-gated candidate proposals and regressions.
4 - Native integration and live health	Validated USB/optional MAVFTP, forwarded telemetry, live alert lifecycle, supported camera/GCS adapters	Device tests and failure recovery; no unauthorized flight/parameter messages; recorder isolation under load.
5 - Release qualification	Installers, accessibility, large-log performance, migration/recovery, reports and operational guide	All mandatory acceptance cases pass; unsupported device capabilities and outstanding limitations are explicit.

Suggested repository areas: desktop shell; UI; engine/collectors; engine/parsers; engine/normalization; engine/clocks; engine/metrics; engine/rules; engine/recommendations; schemas; fixtures; integration tests; docs. Keep DSP/analysis independent of React and the shell.

Record architecture decisions and deviations. If a format/capability cannot be validated without an actual sample, implement the archive/unknown/recovery behavior and identify the exact sample needed. Do not invent an endpoint, unit, device value or successful test.

Performance targets are proposed engineering acceptance targets, not measured claims: responsive UI during a 2 GiB import; cancellation acknowledged within two seconds; bounded memory rather than proportional dictionary growth; cached-window chart queries normally under 300 ms on an agreed reference laptop. Select the laptop/fixture benchmark before using these as release gates.

Package the Python runtime and native dependencies. Validate Windows USB/GCS interactions, macOS signing/notarization and architecture builds, and Linux serial permissions/webview dependencies. Updates must migrate data safely, preserve raw logs and allow recovery from interrupted installation. Offline use must work after installation.

22 Acceptance and validation plan

Use unit/property tests for numerical invariants, real-format fixtures for parser parity, integration tests for collection and manual device tests for hardware behavior.

Test family	Mandatory scenarios
Data identity	Duplicate bytes, duplicate filenames with different bytes, two vehicles in one TLOG, multiple flights in one file, wrong profile and unknown firmware.
Units / schema	Older/newer VIBE formats; fixed-point scale applied once; missing/sentinel values; corrupt/truncated records; unknown units; very large input.
Clocks	FC reboot, counter wrap, PC wall-clock jump, drift, invalid UTC, delayed TLOG delivery and radio time with unknown epoch.
Numerics	Known constant/ramp power; long gaps; negative current; partial windows; battery-monitor duplication; clipping counters starting nonzero/resetting.
DSP / PID	Known sine/noise, missing batch, irregular sampling, aliasing, pre/post provenance; valid/rejected steps; saturation and low-excitation cases.
Coverage	One missing ESC/RPM/temperature channel; stale values; unsupported sensor; partial log; enough data to plot but insufficient to recommend.
Collection	Busy port, armed aircraft, growing log, byte gap, disconnect/resume, changed remote log, bad radio JSON, failed poll, overlapping RF windows, disk full.
Safety of interactions	Inspect every outbound controller message; no arming, mode, mission, parameter writes or log deletion. Read-only source failure cannot stop the controlling GCS.
UX / persistence	Fresh install, offline mode, crash/restart, profile revision, immutable baselines, report reproduction, keyboard/200% zoom and clear synthetic data.
Live observer	Recorder overload, UI crash, stale telemetry, duplicate timestamps, app sleep/resume, alert clear/recur and post-flight revision of a hypothesis.
Hardware validation	Representative Cube and H7 logs; four mapped APD ESCs; both SkyPulse responses; actual GCS forwarding; SIYI capability probe.
Comparison	Matched trials improve known synthetic metrics; confounded payload/wind/time scope blocks causal improvement claims.

Use pinned ArduPilot tools/source and labeled reference datasets for expected outputs, with tolerances justified by decoding, windowing and numerical precision. Review discrepancies instead of fitting tests to the new implementation. Generated fixtures supplement real logs; they do not validate the user's device integration.

For each rule, include positive, negative and abstention cases. The rule is not complete until it refuses a conclusion when prerequisites are absent. Every exported result must retain the evidence and analysis version necessary to reproduce it.

A release is not complete while essential requested capabilities exist only as mock data. A documented hardware dependency is an explicit outstanding item, not a passed acceptance gate. The app may ship a clearly labeled earlier milestone, but must not present it as the finished specification.

23 Open evidence, research limits and next inputs

The architecture can be implemented without further product questions. Device-specific claims and numerical tuning logic need the following evidence before validation.

Needed input	Why it matters	Default until available
One complete same-flight BIN + TLOG + RF JSONL bundle	Establish actual fields, cadence, identities and clock anchors	Implement fixtures and explicit unvalidated adapter state; no claim of real-aircraft validation.
Exact firmware/board and full parameter readbacks for Cube and H7	Pin units, masks, message schemas and tuning applicability	Treat reported version labels as provisional.
Actual air and ground SkyPulse raw responses + firmware	Validate array structure, timestamps, units, rolling windows and API errors	Preserve raw responses; no invented normalized radio metrics.
Verified ESC outpu t/instance/pole and telemetry capability mapping	Avoid double RPM scaling or masking absent ESC channels	Use declared profile mapping, clearly marked pending confirmation.
Accepted original and Brian-based Q22 snapshot files	Preserve the requested baseline pair precisely	Do not synthesize a baseline or overwrite existing files.
Battery datasheet/limits, calibration and usable-energy evidence	Support meaningful voltage, consumption and endurance interpretation	Display measurements and assumptions; withhold unsupported limit or remaining-time claims.

Needed input	Why it matters	Default until available
Ground-station versions, actual log folders and forwarding capability	Design one-button collection and live observation on the real PC	Manual import remains available.
Camera/player diagnostics and a bench latency sample if needed	Separate camera, network and decoding issues	Media import only where supported; latency remains unmeasured.
Reference laptop and flight test protocol	Set fair performance and optimization acceptance criteria	Keep proposed engineering targets labeled as unbenchmarked.

Research distinguishes verified source facts from proposed product/algorithm design. Primary documentation can differ from a particular firmware build. The implementation must prefer actual log definitions and matching source/parameter metadata, with versioned adapters, over an unqualified “current docs” assumption.

The supplied existing Flight Data Review Rev01 established the three-record BIN/TLOG/RF workflow, preservation requirements and the baseline diagnostic order. This handoff extends it into a product architecture. It does not validate aircraft hardware, certify the tune or claim that all requested measurements already exist.

No application implementation is part of this deliverable. Any exploratory code produced before the user redirected this task is excluded and is not an approved foundation for Codex. Start from this specification and validated dependencies/fixtures.

24 Firmware field and parameter research catalog

Representative Copter-4.7.1 fields are listed here to guide implementation. Availability still depends on the build and logging profile. Preserve exact FMT metadata and source revision rather than treating this table as a universal parser schema [R20-R29].

Subsystem	Representative messages / fields	Related context / parameters
Identity / state	VER, MSG, ARM.ArmState/Method/Forced, MODE.ModeNum/Rsn, EV.Id, SPOL	Board/build identity; arming checks; explicit enum tables.
Schema / logging	FMT, FMTU, UNIT, MULT; time-stamped PARM; DSF.Dp/Bytes/FM*	LOG_BACKEND_TYPE, LOG_BITMASK, LOG_DISARMED, LOG_FILE_RATEMAX, LOG_FILE_DSRMROT.
IMU / gyro	IMU.I/EG/EA/T/GH/AH/GHz/AHz; VIBE.IMU/VibeX/Y/Z/Clip; GYR/ACC.SampleUS; ISBH/ISBD	Sensor IDs/orientation; INS_USE*; raw/batch options; exact unit/scaling lineage.
Rates / PID	RATE.RDes/R/PDes/P/YDes/Y/ROut/POut/YOut/AOut/AOutSlew; PID*.P/I/D/FF/DFF/Dmod/SRate/Flags	ATC_RAT_* gains/filters/limits, ATC_ACC_*_MAX, command shaping and sampling configuration.
Actuators	RCOU.C1...; RCO2; MOTB.LiftMax/BatVolt/ThLimit/ThrAvMx/ThrOut/FailFlags	SERVOx_FUNCTION; FRAME_*; MOT_PWM_TYPE, thrust model/voltage/current limits.
Battery / cells	BAT.Inst/Volt/VoltR/Curr/CurrTot/EnrgTot/Temp/Res/RemPct/H/SH; BCL.Instance/V1...V12	BATT_* monitor/calibration/threshold/timer/source settings; physical versus aggregate monitors.
ESC	ESC.Instance/RPM/RawRPM/Volt/Curr/Temp/CTot/MotTemp/Err; ESCX; EDT2 when supported	SERVO_BLH_POLES, SERVO_BLH_BDMASK, SERVO_DSHOT_ESC and applicable serial telemetry configuration.
GPS	GPS.I/Status/GMS/GWk/NSats/HDop/Lat/Lng/Alt/Spd/GCrs/VZ; GPA.HAcc/VAcc/SAcc/YAcc/Delta/RTCM*	GPS source selection/corrections; backend/units; optional receiver interference indicators.
Estimator	XKF3 innovations; XKF4.C/SV/SP/SH/SM/FS/TS/SS/GPS/PI; XKFS sensor selections; XKT timing	EK3_LOG_LEVEL, EK3_SRC*, EK3_IMU_MASK, affinity; FS_EKF_* configuration and implementation.
Compass	MAG.I/MagX/Y/Z/OfsX/Y/Z/MOX/MOY/MOZ/Health; XKFS.MI	COMPASS_* IDs/orientation/calibration/motor compensation; active yaw source.

Subsystem	Representative messages / fields	Related context / parameters
Position / mission	PSCN/PSCE/PSCD desired/target/actual position/velocity/acceleration; PSON/PSOE/PSOD/PSOT; CTUN; CMD; MISE	PSC_*, navigation limits, altitude frame and origin; mission upload versus execution events.
FC power / system	POWR.Vcc/VServo/Flags/AccFlags; MCU; PM.LR/NLon/NL/MaxT/Mem/Load /InE/ErC/ErrL; RTDT	Actual rail specification; SCHED_* configuration; loop rate and board features.
Control / failsafe	RCIN; RCI2.OMask/Flags; MAVC.SS/SC/TS/TC/Cmd/Res/WL; MAV; MODE; ERR; MSG; TLOG ACKs	FS_GCS_*, RC_FS_TIMEOUT, RC_OVERRIDE_TIME, applicable radio/battery/terrain failsafe options.

Do not reduce time-stamped PARM records to the final dictionary when analyzing an earlier maneuver. Retain a configuration timeline and split affected metrics when parameters changed.

Important verified Copter-4.7.1 source traps: BAT.CurrTot is written as consumed mAh. BAT.RemPct=255 and BAT.SH=0 represent unknown values; BAT.Temp=0 is ambiguous without temperature-support context. Raw BCL cell=0 is unavailable; otherwise cell volts=(raw-1)/1000. Raw PM.Load=800 corresponds to 80% load. Check whether the selected parser already applied a transformation before applying it again. XKF4 test outputs are not raw covariance. Modern PSC values use SI and NED/down conventions. CMD records a mission item; MISE indicates execution start, not completion [R21-R29].

The field catalog is a discovery checklist. Only implement a diagnostic after its actual schema, scale, sentinel values and relevant firmware logic have been verified against fixtures. Do not copy a string list into a parser and assume semantic correctness.

25 Logging profiles and evidence acquisition

Provide normal-flight and temporary diagnostic capture plans, generated from the actual aircraft firmware and verified after recording.

Plan	Required evidence	Validation
Normal flight	Events/identity/parameters; IMU health/VIBE; battery; control and actuator state; GPS/EKF; ESC and relevant compass/system health; GCS TLOG; RF recorder	Check the resulting fields, instance count, cadence, gaps, storage and CPU/logging pressure. A configured bit is not proof of collected data.
Filter investigation	Raw/batch gyro with sample timing and known pre/post provenance; RPM and relevant filter parameters	Verify raw encoding, actual sample rate, record continuity and logger load before interpreting FFTs.

Plan	Required evidence	Validation
PID investigation	Representative controlled RATE/PID/actuator/attitude records; filter/noise context; exact parameter timeline	Verify adequate excitation, no invalid saturation/clipping/estimator interval, and enough pre/post response time.
Power investigation	Calibrated voltage/current, cumulative counters, battery identity/state/temperature, mass/payload, motion/phase, available per-ESC data	Cross-check sensors; use matched conditions and preserve gaps; add external electrical measurements where needed.
Navigation investigation	Selected GPS/compass/IMU/barometer/range sources; estimator lanes/innovations/status; position controller and mission/mode events	Preserve coordinate frames and active aiding configuration. Additional EKF/replay logs are a targeted capture choice.
Control-path / video investigation	GCS diagnostic and send/receive capture where supported; actual joystick events; both radios; camera/player/PC metrics	Confirm direction and clock lineage, observer overhead and available latency reference.

The existing Q22 review contains a normal logging configuration and temporary Cube raw-gyro guidance. The app should import that accepted profile as a documented reference after exact firmware verification, not silently push the same numbers to both controllers.

For every proposed logging change, show the name, current value, intended fields, estimated storage/CPU/traffic cost, firmware support, reboot/readback requirements and verification step. More logging is not always better if it causes dropped data or controller overload. Remote DataFlash streaming is optional and bandwidth-tested, not enabled by default on the control link.

When data is missing, distinguish hardware does not measure it, transport does not expose it, logging disabled, stream rate insufficient, recorder failed, file incomplete and decoder unsupported. Each outcome needs a different recovery action.

26 Troubleshooting workspaces

Offer symptom-based entry points that open a useful set of signals and a short evidence-driven decision path.

Symptom	Inspect together	Next discriminating step
Vibration / shaking	VIBE/clipping, gyro orders, rates/PID, outputs/RPM, phase and speed	Separate mechanical noise from control oscillation; inspect hardware or collect missing raw evidence.

Symptom	Inspect together	Next discriminating step
Short flight time / high current	Calibrated power, mass/payload, battery state, speed/climb, RPM/output activity and temperatures	Compare matched phases; test sensor calibration and change one supported factor.
Poor hover / altitude oscillation	Desired/actual vertical motion, clipping, baro/range, EKF, thrust limits and power	Identify estimate error versus actuator/control limitation before changing altitude gains.
Yaw oscillation / drift	Yaw demand/response, heading innovations, CW/CCW output groups, compass/current and limits	Check commanded yaw, allocation/CG, source selection and interference context.
Unexpected mode or failsafe	Mode reasons, active error events, configured timeouts, GCS/RC freshness and aircraft response	Reconstruct the event against the matching firmware; distinguish observed trigger from suspected cause.
Missing / hot / uneven ESC	Mapped instances, RPM, telemetry age, current/voltage/temperature and demand	Verify mapping/capability/wiring and compare under equivalent demand; do not diagnose a stopped motor from missing telemetry.
Video freeze / delay	Camera/player frames, network/decoder load, both radios, traffic demand and TLOG continuity	Separate video-only failure from common link or ground-computer failure; run a controlled latency test if required.
App shows gaps / strange values	Source identity, units/sentinels, clocks/reboots, parser warnings, DSF/PM and recorder state	Correct the observation pipeline before drawing aircraft conclusions.

Primary references

Reviewed 3 October 2026. Pin matching revisions for implementation; current documentation is not a substitute for the aircraft's exact firmware metadata.

[R01] Tauri 2: external binaries / sidecars
https://v2.tauri.app/develop/sidecar/
Native shell and packaged engine capability; architecture choice is our recommendation.

[R02] DuckDB: Parquet overview
https://duckdb.org/docs/stable/data/parquet/overview
Column and time-window query design.

[R03] Apache ECharts
https://echarts.apache.org/en/index.html
Chart library capabilities; app performance must be measured.

[R04] ArduPilot: measuring vibration
https://ardupilot.org/copter/docs/common-measuring-vibration.html
Vibration bands and clipping interpretation.

[R05] ArduPilot: raw IMU logging
https://ardupilot.org/copter/docs/common-raw-imu-logging.html
Raw gyro/filter evidence requirements.

[R06] ArduPilot: harmonic notch filtering
https://ardupilot.org/copter/docs/common-imu-notch-filtering.html
Filter modes and trade-offs; exact masks remain firmware-specific.

[R07] ArduPilot WebTools repository / FilterReview
https://github.com/ArduPilot/WebTools
Reference implementations and license review; pin commits before reuse.

[R08] ArduPilot PIDReview README
https://github.com/ArduPilot/WebTools/blob/main/PIDReview/Readme.md
PID/response analysis prerequisites and rate-only limitations.

[R09] ArduPilot: ESC telemetry
https://ardupilot.org/copter/docs/common-esc-telemetry.html
Telemetry capabilities differ by firmware and protocol.

[R10] DigiQ SkyPulse manual v1.2.0
https://digiq-rf.com/datasheets/DigiQ_RF_SkyPulse_User_Guide_Manual.pdf
Sections 15.2.8-15.2.9; documented API and rolling histories, not hardware-tested here.

[R11] MAVLink: log transfer protocol
https://mavlink.io/en/services/log.html
Controller log retrieval protocol.

[R12] ArduPilot: MAVLink log transfer
https://ardupilot.org/dev/docs/mavlink-log-download.html
Version/capability differences for retrieval.

[R13] MAVLink common messages: SYSTEM_TIME
https://mavlink.io/en/messages/common.html#SYSTEM_TIME
Unix and boot clock fields; validity must be checked.

[R14] ArduPilot: onboard log message catalog
https://ardupilot.org/copter/docs/logmessages.html
Representative field discovery; use matching source/FMT for exact build.

[R15] ArduPilot: log download and analysis
https://ardupilot.org/copter/docs/common-downloading-and-analyzing-data-logs-in-mission-planner.html
Logging configuration and dropped-message diagnostics.

[R16] ArduPilot pymavlink DFReader
https://github.com/ArduPilot/pymavlink/blob/master/DFReader.py
Decoding/scaling/time behavior; pin parser revision.

[R17] ArduPilot: diagnosing problems using logs
https://ardupilot.org/copter/docs/common-diagnosing-problems-using-logs.html
Health review context.

[R18] ArduPilot: tuning process
https://ardupilot.org/copter/docs/tuning-process-instructions.html
Mechanical/filter/control tuning order.

[R19] ArduPilot: SIYI camera/gimbal integration
https://ardupilot.org/copter/docs/common-siyi-zr10-gimbal.html
Model/firmware-dependent integration and media retrieval.

[R20] ArduPilot Copter-4.7.1 source tree
https://github.com/ArduPilot/ardupilot/tree/Copter-4.7.1
Version-specific implementation authority when the aircraft matches this tag.

[R21] Copter-4.7.1: logger structures
https://raw.githubusercontent.com/ArduPilot/ardupilot/Copter-4.7.1/libraries/AP_Logger/LogStructure.h
Schema and event/command fields.

[R22] Copter-4.7.1: IMU structures
https://raw.githubusercontent.com/ArduPilot/ardupilot/Copter-4.7.1/libraries/AP_InertialSensor/LogStructure.h
Modern VIBE, raw and batch fields.

[R23] Copter-4.7.1: EKF outputs
https://raw.githubusercontent.com/ArduPilot/ardupilot/Copter-4.7.1/libraries/AP_NavEKF3/AP_NavEKF3_Outputs.cpp
Test metric transformations and meaning.

[R24] Copter-4.7.1: EKF structures
https://raw.githubusercontent.com/ArduPilot/ardupilot/Copter-4.7.1/libraries/AP_NavEKF3/LogStructure.h
Lane/source/innovation metadata.

[R25] Copter-4.7.1: battery logging
https://raw.githubusercontent.com/ArduPilot/ardupilot/Copter-4.7.1/libraries/AP_BattMonitor/AP_BattMonitor_Logging.cpp
Consumption units and sentinel/encoding behavior.

[R26] Copter-4.7.1: ESC structures
https://raw.githubusercontent.com/ArduPilot/ardupilot/Copter-4.7.1/libraries/AP_ESC_Telem/LogStructure.h
ESC schemas; verify actual installed firmware.

[R27] Copter-4.7.1: scheduler
https://raw.githubusercontent.com/ArduPilot/ardupilot/Copter-4.7.1/libraries/AP_Scheduler/AP_Scheduler.cpp
Scheduler metrics and load encoding.

[R28] Copter-4.7.1: attitude/position structures
https://raw.githubusercontent.com/ArduPilot/ardupilot/Copter-4.7.1/libraries/AC_AttitudeControl/LogStructure.h
Modern PSC, RATE and control units.

[R29] Copter-4.7.1: mission structures
https://raw.githubusercontent.com/ArduPilot/ardupilot/Copter-4.7.1/libraries/AP_Mission/LogStructure.h
Mission logging semantics.

[R30] ArduPilot: MAVFTP
https://ardupilot.org/dev/docs/mavlink-mavftp.html
Optional transport and version-specific paths.

[R31] EdgeTX: special functions / SD Logs
https://manual.edgetx.org/color-radios/model-settings/special-functions
Optional radio-side CSV when SD logging is enabled; available values depend on inputs.

[R32] ArduPilot batch sampler implementation
https://github.com/ArduPilot/ardupilot/blob/master/libraries/AP_InertialSensor/BatchSampler.cpp
Batch scale/encoding reference; pin matching revision before implementation.

[R33] Copter-4.7.1: EKF failsafe logic
https://raw.githubusercontent.com/ArduPilot/ardupilot/Copter-4.7.1/ArduCopter/ekf_check.cpp
Configured threshold and filtered/state-dependent failsafe behavior.